System Testing, Maintenance Planning, and Reporting for an RPM Transmission-Gap Detection System

Anam Kazani

Department of Information Technology, University of the People

MSIT 5910-01: MSIT Capstone Project

Dr. Sirisha

October 21, 2026

System Testing, Maintenance Planning, and Reporting for an RPM Transmission-Gap Detection System

I am presenting my capstone system, in which remote patient monitoring (RPM) devices are flagged when they become inactive compared to the usual pattern of their activity, and patients who need nursing outreach are identified. Unit 6 merged its four modules into one application, version 0.3.0. This unit carries out tests of the whole system, addresses errors identified by the tests in version 0.4.0, and creates the strategy for system support after deployment. It is important to stress that all the data is synthetic.

Part A: System Testing and Documentation

System Testing

The 21 unit tests from Unit 5 still check each module under test. A further testing source file, test_system.py, was built for system-level testing, where another 25 test cases at four additional levels, plus regression tests, were created by me. In the integration tests, I tested the modules and database based on their ability to exchange data. The system tests checked the application as a black box by sending it real requests through HTTP to evaluate its functional and non-functional capabilities, which is how Hrupa (2024) and GeeksforGeeks (2025b) describe system testing. Acceptance tests directly assessed the Unit 3 functional requirements from the perspective of the nurse. Regression tests confirmed that the Unit 6 accuracy and detection times did not change, and performance tests checked speed and stability in an environment that resembles production, as BrowserStack (2025) advises.

The first run against version 0.3.0 passed 41 tests and failed four (Figures 1 and 2). One test, for the FR-8 cohort report, is marked as an expected failure because that report has not been built yet.

Figure 1

First Test Run Against Version 0.3.0

```
tests/test_gap_detection.py::TestBlackBox::test_S01_daily_device_quiet_10h_is_normal PASSED                                                    [  2%]
tests/test_gap_detection.py::TestBlackBox::test_S02_quiet_two_usual_gaps_is_flagged PASSED                                                     [  4%]
tests/test_gap_detection.py::TestBlackBox::test_S03_quiet_47h_is_not_flagged PASSED                                                            [  6%]
tests/test_gap_detection.py::TestBlackBox::test_S04_rural_device_quiet_3_5_days_is_normal PASSED                                               [  8%]
tests/test_gap_detection.py::TestBlackBox::test_S05_twice_daily_device_quiet_30h_is_flagged PASSED                                             [ 10%]
tests/test_gap_detection.py::TestBlackBox::test_S06_needs_a_full_day_of_silence PASSED                                                         [ 13%]
tests/test_gap_detection.py::TestBlackBox::test_S07_burst_of_readings_is_one_session PASSED                                                    [ 15%]
tests/test_gap_detection.py::TestBlackBox::test_I01_only_consented_patients_and_3_fields PASSED                                                [ 17%]
tests/test_gap_detection.py::TestBlackBox::test_I02_unreadable_or_future_time_not_stored PASSED                                                [ 19%]
tests/test_gap_detection.py::TestBlackBox::test_Q01_own_program_only_highest_score_first PASSED                                                [ 21%]
tests/test_gap_detection.py::TestBlackBox::test_Q02_unknown_user_gets_empty_queue PASSED                                                       [ 23%]
tests/test_gap_detection.py::TestBlackBox::test_L01_cannot_log_call_for_other_program PASSED                                                   [ 26%]
tests/test_gap_detection.py::TestBlackBox::test_L02_unknown_outcome_refused PASSED                                                             [ 28%]
tests/test_gap_detection.py::TestBlackBox::test_E01_flags_match_synthetic_answer_key PASSED                                                    [ 30%]
tests/test_gap_detection.py::TestWhiteBox::test_W01_no_readings_counts_from_enrollment PASSED                                                  [ 32%]
tests/test_gap_detection.py::TestWhiteBox::test_W02_pattern_needs_five_gaps PASSED                                                             [ 34%]
tests/test_gap_detection.py::TestWhiteBox::test_W03_reading_after_as_of_is_ignored PASSED                                                      [ 36%]
tests/test_gap_detection.py::TestWhiteBox::test_W04_quiet_device_beside_working_one PASSED                                                     [ 39%]
tests/test_gap_detection.py::TestWhiteBox::test_W05_resent_reading_stored_once PASSED                                                          [ 41%]
tests/test_gap_detection.py::TestWhiteBox::test_W06_logged_call_leaves_queue_and_cannot_change PASSED                                          [ 43%]
tests/test_gap_detection.py::TestWhiteBox::test_W07_demo_runs_to_the_end PASSED                                                                [ 45%]
tests/test_system.py::TestIntegration::test_IT01_login_then_queue_in_score_order PASSED                                                        [ 47%]
tests/test_system.py::TestIntegration::test_IT02_queue_shows_what_scoring_saved PASSED                                                         [ 50%]
tests/test_system.py::TestIntegration::test_IT03_logged_call_is_saved_and_leaves_queue PASSED                                                  [ 52%]
tests/test_system.py::TestIntegration::test_IT04_failed_scoring_leaves_no_half_saved_data PASSED                                               [ 54%]
tests/test_system.py::TestIntegration::test_IT05_reading_the_same_file_twice_adds_nothing PASSED                                               [ 56%]
tests/test_system.py::TestSystem::test_ST01_full_walkthrough_passes_all_checks PASSED                                                          [ 58%]
tests/test_system.py::TestSystem::test_ST02_health_check_reports_version PASSED                                                                [ 60%]
tests/test_system.py::TestSystem::test_ST03_unknown_address_returns_404 PASSED                                                                 [ 63%]
tests/test_system.py::TestSystem::test_ST04_missing_or_fake_token_returns_401 PASSED                                                           [ 65%]
tests/test_system.py::TestSystem::test_ST05_other_programs_patient_returns_403 PASSED                                                          [ 67%]
tests/test_system.py::TestSystem::test_ST06_body_that_is_not_a_json_object_returns_400 FAILED                                                  [ 69%]
tests/test_system.py::TestSystem::test_ST07_call_without_patient_returns_400 FAILED                                                            [ 71%]
tests/test_system.py::TestSystem::test_ST08_times_with_a_time_zone_are_stored FAILED                                                           [ 73%]
tests/test_system.py::TestSystem::test_ST09_patient_data_is_never_cached PASSED                                                                [ 76%]
tests/test_system.py::TestAcceptance::test_AT01_FR2_only_three_fields_are_kept PASSED                                                          [ 78%]
tests/test_system.py::TestAcceptance::test_AT02_FR3_no_consent_never_reaches_a_nurse PASSED                                                    [ 80%]
tests/test_system.py::TestAcceptance::test_AT03_FR5_every_row_explains_itself PASSED                                                           [ 82%]
tests/test_system.py::TestAcceptance::test_AT04_FR6_each_nurse_sees_only_her_program PASSED                                                    [ 84%]
tests/test_system.py::TestAcceptance::test_AT05_FR7_a_call_can_be_logged_only_once FAILED                                                      [ 86%]
tests/test_system.py::TestAcceptance::test_AT06_FR8_cohort_report_exists XFAIL (FR-8 cohort report is not built yet (planned for a later version)) [ 89%]
tests/test_system.py::TestRegression::test_RT01_accuracy_on_240_patients_is_unchanged PASSED                                                   [ 91%]
tests/test_system.py::TestRegression::test_RT02_detection_times_are_unchanged PASSED                                                           [ 93%]
tests/test_system.py::TestPerformance::test_PT01_NFR3_4800_patients_scored_well_within_an_hour PASSED                                          [ 95%]
tests/test_system.py::TestPerformance::test_PT02_queue_answers_95_percent_within_200_ms PASSED                                                 [ 97%]
tests/test_system.py::TestPerformance::test_PT03_50_requests_at_once_all_succeed PASSED                                                        [100%]
```

Figure 2

Why the Four Tests Failed

```
rehmankazani@Mac rpm-gap-detection % cd before-fixes && ../.venv/bin/python -m pytest -v
tests/test_system.py:170: AssertionError
_____________________________________________ TestSystem.test_ST08_times_with_a_time_zone_are_stored _____________________________________________

self = <test_system.TestSystem object at 0x106b47950>

    def test_ST08_times_with_a_time_zone_are_stored(self):
        as_of = datetime(2026, 10, 1, 9, 0)
        reading = (as_of - timedelta(hours=30)).astimezone(timezone.utc)   # e.g. ...+00:00
>       assert gd.clean_time(reading.isoformat(), as_of) == (
            as_of - timedelta(hours=30)).isoformat(timespec="seconds")
E       AssertionError: assert None == '2026-09-30T03:00:00'
E        +  where None = <function clean_time at 0x106b6dc70>('2026-09-30T08:00:00+00:00', datetime.datetime(2026, 10, 1, 9, 0))
E        +    where <function clean_time at 0x106b6dc70> = gd.clean_time
E        +    and   '2026-09-30T08:00:00+00:00' = <built-in method isoformat of datetime.datetime object at 0x106921f20>()
E        +      where <built-in method isoformat of datetime.datetime object at 0x106921f20> = datetime.datetime(2026, 9, 30, 8, 0, tzinfo=datetime.timezone.utc
.isoformat
E        +  and   '2026-09-30T03:00:00' = <built-in method isoformat of datetime.datetime object at 0x1069218c0>(timespec='seconds')
E        +    where <built-in method isoformat of datetime.datetime object at 0x1069218c0> = (datetime.datetime(2026, 10, 1, 9, 0) - datetime.timedelta(days=1,
econds=21600)).isoformat
E        +      where datetime.timedelta(days=1, seconds=21600) = timedelta(hours=30)

tests/test_system.py:175: AssertionError
_____________________________________ TestAcceptance.test_AT05_FR7_a_call_can_be_logged_only_once _____________________________________

self = <test_system.TestAcceptance object at 0x106ad32f0>
app = ('http://127.0.0.1:51021', <sqlite3.Connection object at 0x106b4ac50>, [{'patient_id': 'P-1001', 'device_id': 'DEV-400...sent': 'Y', ...}, {'patient_id':
P-1006', 'device_id': 'DEV-4006', 'program': 'Program A', 'consent': 'Y', ...}, ...])

    def test_AT05_FR7_a_call_can_be_logged_only_once(self, app):
        base = app[0]
        token = sign_in(base)
        patient = gd.call(base, "GET", "/api/queue", token=token)[1]["patients"][0]["patient_id"]
        body = {"patient_id": patient, "outcome": "resolved"}
        assert gd.call(base, "POST", "/api/calls", body, token)[0] == 201
>       assert gd.call(base, "POST", "/api/calls", body, token)[0] == 409
E       assert 201 == 409

tests/test_system.py:224: AssertionError
=========================================================== short test summary info ============================================================
FAILED tests/test_system.py::TestSystem::test_ST06_body_that_is_not_a_json_object_returns_400 - AssertionError: assert 'no reply' == 400
FAILED tests/test_system.py::TestSystem::test_ST07_call_without_patient_returns_400 - assert 403 == 400
FAILED tests/test_system.py::TestSystem::test_ST08_times_with_a_time_zone_are_stored - AssertionError: assert None == '2026-09-30T03:00:00'
FAILED tests/test_system.py::TestAcceptance::test_AT05_FR7_a_call_can_be_logged_only_once - assert 201 == 409
========================================================= 4 failed, 41 passed, 1 xfailed in 7.68s ==========================================================
```

I fixed each defect in version 0.4.0 (Figure 3) and ran every test again. All 45 tests now pass, with the one expected failure (Figure 4). The performance tests ingested and scored 4,800 patients in 0.79 seconds, and the queue API answered 95% of 100 requests within 1.0 millisecond (Figure 5).

Figure 3

Fix for Defects D2 and D4 in log_outcome()

```
class AlreadyLogged(Exception):
    """A call outcome is already saved for this patient."""


def log_outcome(db, nurse, patient_id, outcome):
    """Save what the call found. Nurses can only log their own patients,
    and only once per patient, because a logged call is final."""
    if not patient_id:
        raise ValueError("patient_id is required")
    if outcome not in OUTCOMES:
        raise ValueError(f"unknown outcome: {outcome}")
    row = db.execute("SELECT program FROM patients WHERE patient_id = ?",
                     (patient_id,)).fetchone()
    if row is None or row[0] != NURSES.get(nurse):
        raise PermissionError(f"{nurse} cannot log calls for {patient_id}")
    if db.execute("SELECT 1 FROM calls WHERE patient_id = ?", (patient_id,)).fetchone():
        raise AlreadyLogged(f"a call is already logged for {patient_id}")
    db.execute("INSERT INTO calls VALUES (?, ?, ?, ?)",
               (patient_id, outcome, nurse, datetime.now().isoformat(" ", "minutes")))

```

Figure 4

Second Test Run After the Fixes (Version 0.4.0)

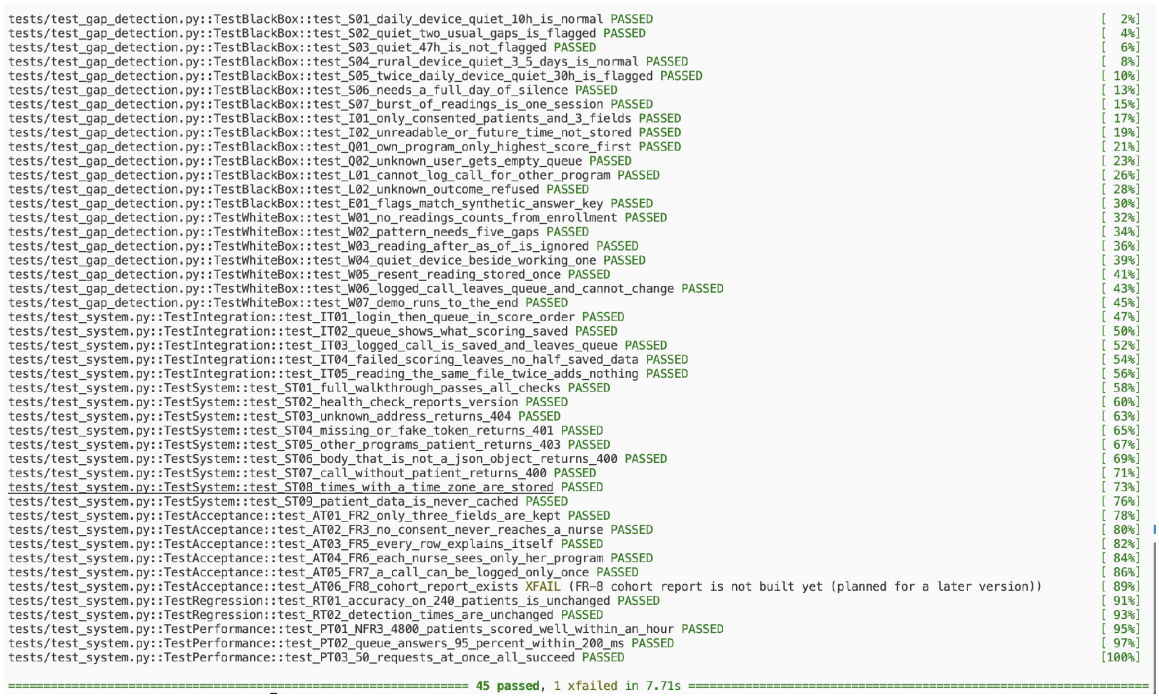
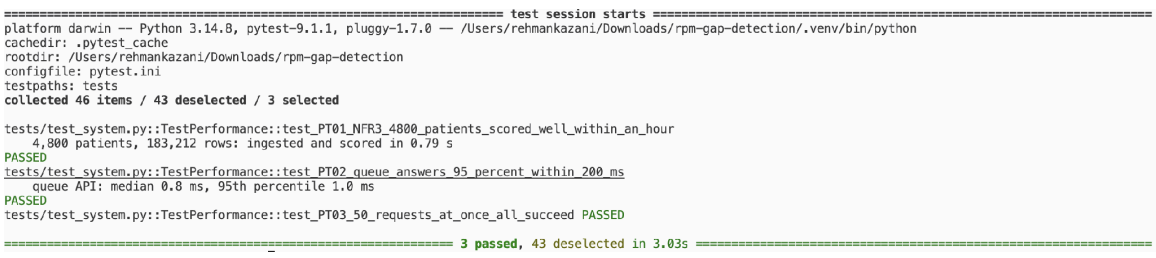


Figure 5

Performance Test Output



Test Documentation

Testing aimed at determining whether version 0.3.0 satisfies the Unit 3 specifications as a complete system before getting ready for deployment. It included testing of all four modules, the web API, and the database with synthetic data on Python 3.14 on a Mac. Following GeeksforGeeks (2025b), the process included the following steps: writing test cases from requirements, preparing test data, executing tests, recording defects, correcting them, and

executing regression tests once again. Every test case is listed by its ID in Figures 1 and 4, and the results and defects are summarized in Tables 1 and 2, respectively.

Table 1

Test Summary Report

Level	**Tests**	**What it checks**	**First run (v0.3.0)**	**After fixes (v0.4.0)**
Unit	21	Each module alone	21 passed	21 passed
Integration	5	Modules and database together	5 passed	5 passed
System	9	Whole running application over HTTP	6 passed, 3 failed	9 passed
Acceptance	6	Functional requirements FR-2 to FR-8	4 passed, 1 failed, 1 expected failure	5 passed, 1 expected failure
Regression	2	Unit 6 results unchanged	2 passed	2 passed
Performance	3	Speed and stability	3 passed	3 passed
Total	46		41 passed, 4 failed, 1 expected failure	45 passed, 1 expected failure

Table 2

Defects Found and Fixed

ID	Test	Problem in version 0.3.0	Fix in version 0.4.0
D1	ST06	A request body that was not a JSON object, such as [], crashed the request, and the server closed the connection without replying	The API checks the body and answers 400
D2	ST07	Logging a call with no patient ID returned 403 instead of 400	A patient ID is required
D3	ST08	Reading times with a time zone, such as +00:00, were rejected, so a vendor sending UTC times would lose every reading	Times with a zone are converted to local time
D4	AT05	The same call could be logged twice, giving one patient two outcomes	A second call returns 409

While fixing these, I also made the dashboard escape apostrophes in patient IDs. Some limitations remain documented rather than fixed. The FR-8 cohort report is missing, usability has not been tested with nurses, the database is kept in memory, and devices that normally send every three days still take a median of 142 hours to flag.

Evaluation

The unit tests were successful; however, system testing discovered four defects. The defects occurred on the boundaries where the system interacts with the vendor files and client requests, a situation where there is no single module accountable. Hrupa (2024) claims that system testing is the final structured check before users judge a product; D3 shows exactly that, as on the first day of using a new vendor it would have simply rejected every real reading. The

testing carried out indeed confirmed that the changes had not affected the results, as both precision and recall remain at 1.00 and the times of detection stayed the same—25,49 and 142 hours accordingly. The fact that every case is written according to some requirement allows tracing back all results to the original design, meaning that the system would be accepted and supported more easily.

Part B: System Maintenance Plan

Maintenance Strategy

Thales (n.d.) describes four types of maintenance. Corrective maintenance fixes faults, such as D1 to D4, and is the most urgent here because a fault in scoring could hide a silent device. Adaptive maintenance adjusts the system to changes around it, such as a new vendor export format, a new Python version, or the organization's identity system. D3 was an adaptive problem found early. Perfective maintenance adds what users ask for, such as showing silences in days, a link to the patient's phone number, the FR-8 report, and a four-day cap for slow senders. Preventive maintenance deals with latent faults before they cause harm (GeeksforGeeks, 2025a), such as updating Python and rerunning the full test suite. Table 3 sets the schedule.

Table 3

Monitoring and Maintenance Schedule

When	Activity	Type
Every scoring pass	Check /api/health and record ingestion counts	Preventive
Daily	Review rejected and duplicate rows; a jump means the vendor file changed	Corrective, adaptive

When	Activity	Type
Weekly	Review open bug reports and nurse requests	Corrective, perfective
Monthly	Update Python, run all 46 tests, check response times	Preventive
Quarterly	Review the threshold, flag rates, and false alarms with nurses	Perfective

Maintenance Documentation

Every change goes through the steps in Thales (n.d.), which are to identify, analyze, design, implement, test, accept, and deliver. Bug reports and enhancement requests are turned into GitHub issues with steps to reproduce the issue. For bugs, I start by creating a test that fails, as I did with D1 to D4, fix the bug on a feature branch created from development, and finally merge it when it passes all tests. Versions follow the structure major.minor.patch, where bug fixes change the last number and new features the middle number, and the changelog documents each release.

Version control and configuration management work side by side in this plan. Each tagged release on main is a restore point in case an update runs into issues, which Davies (2024) describes as the flexibility and accountability of version history. Controlled change is an aim of configuration management (Summers, 2020), so the flag threshold RPM_FLAG_AT can only be changed through a properly reviewed release, as this threshold determines which patients are called. A GitHub Actions workflow should run all tests for each pull request.

Risk Identification and Mitigation

Table 4 lists the main risks after deployment and how each will be handled.

Table 4

Post-Deployment Risks

Risk	Mitigation	Contingency
Security: sign-in tokens never expire in the prototype	Use the organization's identity system with expiring sessions and HTTPS	Clear all tokens and review the call log
Security: outdated Python with known vulnerabilities	Monthly updates, tested before release	Roll back to the previous release
Data inconsistency: the vendor changes its file or time format	Daily review of rejected rows and format tests like ST08	Pause, fix, and reload the file, which adds no duplicates
Data loss: call outcomes kept only in memory	Database file on an encrypted disk with nightly backups	Restore the last backup
Usability: nurses stop trusting the queue	Keep false alarms low and review the queue with nurses quarterly, since repeated alerts cause alert fatigue (Ancker et al., 2017)	Fall back to the monthly report

References

Ancker, J. S., Edwards, A., Nosal, S., Hauser, D., Mauer, E., & Kaushal, R. (2017). Effects of workload, work complexity, and repeated alerts on alert fatigue in a clinical decision support system. *BMC Medical Informatics and Decision Making, 17*(1), Article 36. https://doi.org/10.1186/s12911-017-0430-8

BrowserStack. (2025, May 29). *What is system testing? (Examples, use cases, types)*. https://www.browserstack.com/guide/what-is-system-testing

Davies, S. (2024). *Blueprints: Creating, describing, and implementing designs for larger-scale software projects* (Version 2.5). University of Mary Washington. https://open.umn.edu/opentextbooks/textbooks/blueprints-creating-describing-and-implementing-designs-for-larger-scale-software-projects-davis

GeeksforGeeks. (2025a, September 29). *Software maintenance – Software engineering*. https://www.geeksforgeeks.org/software-engineering/software-engineering-software-maintenance/

GeeksforGeeks. (2025b, July 22). *System testing – Software testing*. https://www.geeksforgeeks.org/software-testing/system-testing/

Hrupa, A. (2024, March 19). *What is system testing in software testing*. Testfort. https://testfort.com/blog/what-is-system-testing-in-software-testing

Summers, B. L. (2020). *Effective methods for software engineering*. Auerbach Publishers.

Thales. (n.d.). *What is a software maintenance process? 4 types of software maintenance*. https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance